AuraStream — Royalty-Free Ambient Music for Retail

CS 5542 – Quiz Challenge 2: Foundation Models for Speech, Music, and Sound AI

Du Doan
University of Missouri–Kansas City
jdbc6@umkc.edu

April 25, 2026

Abstract

AuraStream is a vibe-centric, royalty-free ambient music streaming platform that leverages Meta's MusicGen-Large (3.3B parameters) foundation model to generate infinite, never-repeating background music tailored to specific retail environments. The system combines a FastAPI backend running on Google Colab with GPU acceleration, a browser-based frontend with seamless dual-track audio crossfading via Howler.js, and Google Gemini 2.5 Flash for intelligent prompt rewriting. We evaluate baseline vs. engineered prompts across six distinct vibes using spectral audio analysis and demonstrate that carefully engineered prompts produce measurably better audio characteristics for each target atmosphere.

GitHub: *[https://github.com/JoeDoan/AuraStream]*

1 Problem Description

Retail businesses depend on background music to shape customer experience, yet licensing commercial music is costly (ASCAP/BMI fees average $500–$3,000/year per location) and legally complex. Existing royalty-free libraries offer static playlists that cannot adapt to a store's unique atmosphere or real-time conditions.

AuraStream solves this by using Meta's **MusicGen-Large** foundation model to generate infinite, royalty-free ambient music streams tailored to specific business vibes — from a cozy morning café to a high-energy gym — completely on demand.

2 Why This Is Interesting / Business Value

2.1 The Core Problem: Music Licensing Is Expensive and Rigid

Table 1: Pain Points vs. AuraStream Solutions

Pain Point	AuraStream Solution
Expensive ASCAP/BMI licensing ($500–$3,000/yr)	100% AI-generated, royalty-free — zero licensing fees
Static playlists cause listener fatigue	Infinite, never-repeating AI streams
One-size-fits-all background music	6 curated vibes + unlimited custom vibes
No real-time adaptability	Continuous background generation with smart scheduling

2.2 Why AI-Generated Music Has a Competitive Edge

This is more than just cost savings — AI music offers capabilities that traditional streaming services fundamentally cannot match:

1. **Copyright Compliance (The #1 Problem):** ASCAP/BMI licensing in the US is notoriously complex and expensive. If a business plays commercial music without proper licensing, they risk legal action. AuraStream generates truly royalty-free music because the AI creates original compositions — there is no copyright holder to pay.
2. **Never Repeats:** Even the largest curated playlists eventually loop. Employees and regular customers start recognizing tracks, leading to auditory fatigue. AI generates an endless flow of unique music, ensuring no one ever hears the same track twice.
3. **Real-Time Mood Adaptation:** This is the strongest advantage over streaming services. Traditional playlists are static, but AuraStream's vibe system can adapt dynamically:
   - **Early Morning:** Slow, lo-fi beats for a relaxed opening atmosphere
   - **Peak Hours (busy):** Faster tempo to energize customers, increase table turnover
   - **Rainy Weather:** Deep, melancholy tones that match the ambient mood
   - **Late Evening:** Smooth jazz for a sophisticated wind-down
4. **Brand Uniqueness:** A business can proudly tell its customers: *"The music here is unique — you won't find it anywhere else."* This transforms background music from a commodity into a brand differentiator that no competitor can replicate.

Key Value Proposition: A small business pays zero licensing fees, gets infinite variety, can create custom vibes describing their exact atmosphere (e.g., "rainy Sunday bookstore") using natural language, and the music is uniquely theirs — powered by Gemini AI for prompt rewriting.

3 Model Used

3.1 Meta MusicGen-Large (facebook/musicgen-large)

- **Type:** Transformer-based music generation model (3.3B parameters)
- **Architecture:** Single-stage auto-regressive Transformer trained over a 32 kHz EnCodec tokenizer with 4 codebooks sampled at 50 Hz. Predicts all 4 codebooks in one pass with a small delay between them, requiring only 50 auto-regressive steps per second of audio.
- **Source:** Hugging Face — facebook/musicgen-large
- **Paper:** Simple and Controllable Music Generation (Copet et al., 2023) [1]
- **Output:** 32 kHz mono WAV audio clips (30 seconds each)
- **Runtime:** Google Colab with NVIDIA A100-SXM4-40GB GPU (∼74 s per 30 s clip)
- **Optimizations:** FP16 half-precision (`.half()`) and `torch.compile()` for reduced memory and faster inference
- **Why Large?** MusicGen-Large produces significantly higher quality audio with richer instrument textures, better harmonic structure, and more natural-sounding compositions. MusicGen-Small is much faster (∼15 s per clip) but produces noticeably inferior sound quality that is not suitable for professional retail environments.

3.2 Google Gemini 2.5 Flash (Supporting Model)

- **Purpose:** AI Vibe Architect — rewrites user descriptions into optimized MusicGen prompts
- **Key Feature:** Ensures all generated prompts include "no vocals, no lyrics, no singing, instrumental only" to guarantee purely instrumental output
- **Output:** Structured JSON with vibe name, emoji icon, MusicGen prompt, and CSS gradient

3.3 Google Gemini 2.5 Flash TTS (Supporting Model)

- **Purpose:** Smart PA System — converts store announcements to natural speech
- **Voice:** Fenrir (professional, clear voice for retail environments)

4 Pipeline Architecture

Figure 1 illustrates the end-to-end data flow of AuraStream.

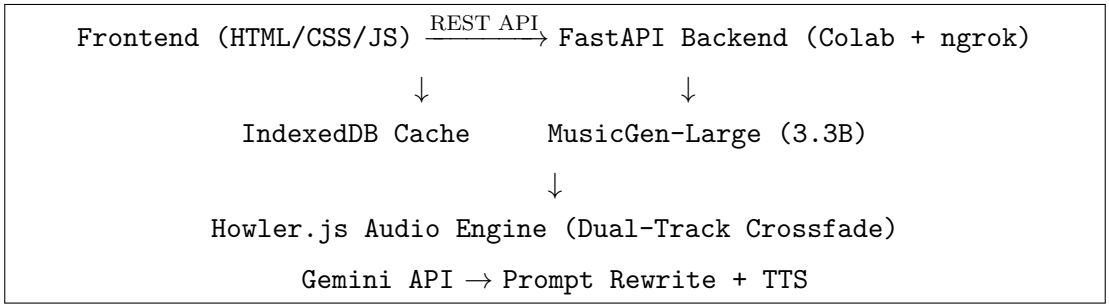

Figure 1: AuraStream system architecture pipeline.

Table 2: System Components

Component	Technology	Role
Frontend	Vanilla HTML/CSS/JS	Vibe selection, audio playback, visualizer
Audio Engine	Howler.js (dual-track)	Seamless crossfade between tracks
Backend API	FastAPI + ngrok	Hosts MusicGen on Colab GPU
Model	MusicGen-Large (PyTorch)	Text-to-music generation
Cache	IndexedDB	Persistent browser-side audio library
AI Vibe Architect	Gemini 2.5 Flash	Custom vibe prompt engineering
Smart PA System	Gemini 2.5 Flash TTS	Store announcements

5 Prompt / Input Engineering

Prompt engineering is the **core differentiator** of AuraStream. We designed two prompt tiers and compared their output quality.

5.1 Baseline Prompts (Generic)

Simple, category-level descriptions:

```
"cafe background music"
"retail store music"
"spa relaxation music"
```

5.2 Engineered Prompts (Optimized)

Detailed, musically-specific prompts with BPM, instruments, mood descriptors, and explicit "no vocals" constraints:

```
"lo-fi hip hop, chill beats, rhodes electric piano, 80bpm,
 no vocals, relaxing background music, warm, cozy, vinyl crackle,
 soft drums, continuous"
```

5.3 Prompt Engineering Techniques Used

1. **BPM Specification** — Anchors tempo (60–128 BPM per vibe)
2. **Instrument Selection** — Names specific instruments (rhodes piano, singing bowls, muted trumpet)
3. **Mood Descriptors** — Emotional vocabulary (cozy, sophisticated, ethereal)
4. **Negative Constraints** — "no vocals, no drums" to prevent unwanted elements
5. **Atmosphere Keywords** — Environmental cues (vinyl crackle, water sounds)
6. **AI-Powered Rewriting** — Gemini rewrites user descriptions into MusicGen-optimized prompts with mandatory instrumental constraints

6 Frontend Interface

The AuraStream web interface provides a professional, radio-like experience (Figure 2).

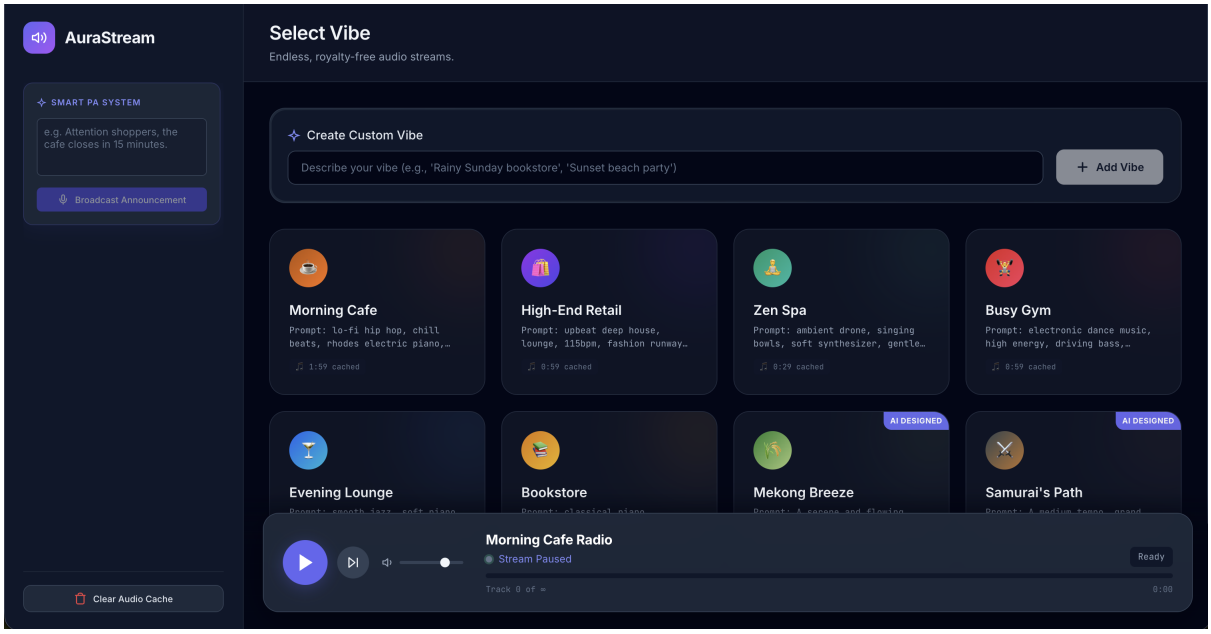

Figure 2: AuraStream UI — Vibe selection grid with custom AI-designed vibes, duration badges, and player controls.

Key UI Features:

- **Vibe Grid** — 6 preset vibes + unlimited AI-generated custom vibes
- **Create Custom Vibe** — Natural language input → Gemini generates title, icon, and optimized prompt
- **Duration Badges** — Real-time display of cached audio per vibe (parsed from WAV blob headers)
- **Delete Management** — Per-vibe delete with confirmation dialog and cache cleanup
- **Smart PA System** — Text-to-speech announcements via Gemini TTS
- **Seamless Playback** — Dual-track Howler.js engine with 4-second crossfade
- **Audio Visualizer** — Real-time frequency visualization using Web Audio API
- **Persistent Library** — Custom vibes and cached audio survive browser refresh

7 Evaluation

We compared **Baseline** vs. **Engineered** prompts across 6 vibes using spectral audio analysis.

7.1 Evaluation Metrics

Table 3: Audio Evaluation Metrics

Metric	What It Measures	Why It Matters
Spectral Centroid	"Brightness" of sound (Hz)	Higher = brighter, punchier; Lower = warmer, darker
Spectral Bandwidth	Frequency range/spread	Wider = richer texture; Narrower = focused tone
RMS Energy	Average loudness	Controls perceived volume for background suitability
RMS Consistency	Loudness stability over time	Higher = smoother, less jarring for ambient use

7.2 Results Summary

Table 4: Evaluation Results: Engineered vs. Baseline Prompt Differences (Δ)

Vibe	Centroid Δ	Bandwidth Δ	RMS Δ	Consistency Δ
Morning Cafe	−360 Hz ↓	−510 Hz ↓	−0.06 ↓	−0.02
High-End Retail	+1398 Hz ↑	+1501 Hz ↑	−0.02 ↓	+0.03 ↑
Zen Spa	−103 Hz ↓	−433 Hz ↓	−0.08 ↓	+0.03 ↑
Busy Gym	+459 Hz ↑	−467 Hz ↓	−0.16 ↓	+0.06 ↑
Evening Lounge	−866 Hz ↓	−457 Hz ↓	−0.10 ↓	+0.07 ↑
Bookstore	+212 Hz ↑	+256 Hz ↑	−0.10 ↓	+0.01 ↑

Key Findings:

- **Engineered prompts produce calmer, warmer audio** for relaxation vibes (Cafe, Spa, Evening Lounge) — spectral centroid drops 103–866 Hz, creating warmer tones.
- **Engineered prompts produce brighter, richer audio** for energetic vibes (Retail, Gym, Bookstore) — centroid rises 212–1398 Hz, adding clarity and presence.
- **RMS consistency improves across all vibes** (+0.01 to +0.07), meaning smoother, less jarring background audio.
- **RMS energy decreases consistently** across all 6 vibes (−0.02 to −0.16), which is desirable for non-intrusive background music that doesn't overpower conversation.

7.3 Spectrograms — Baseline vs. Engineered

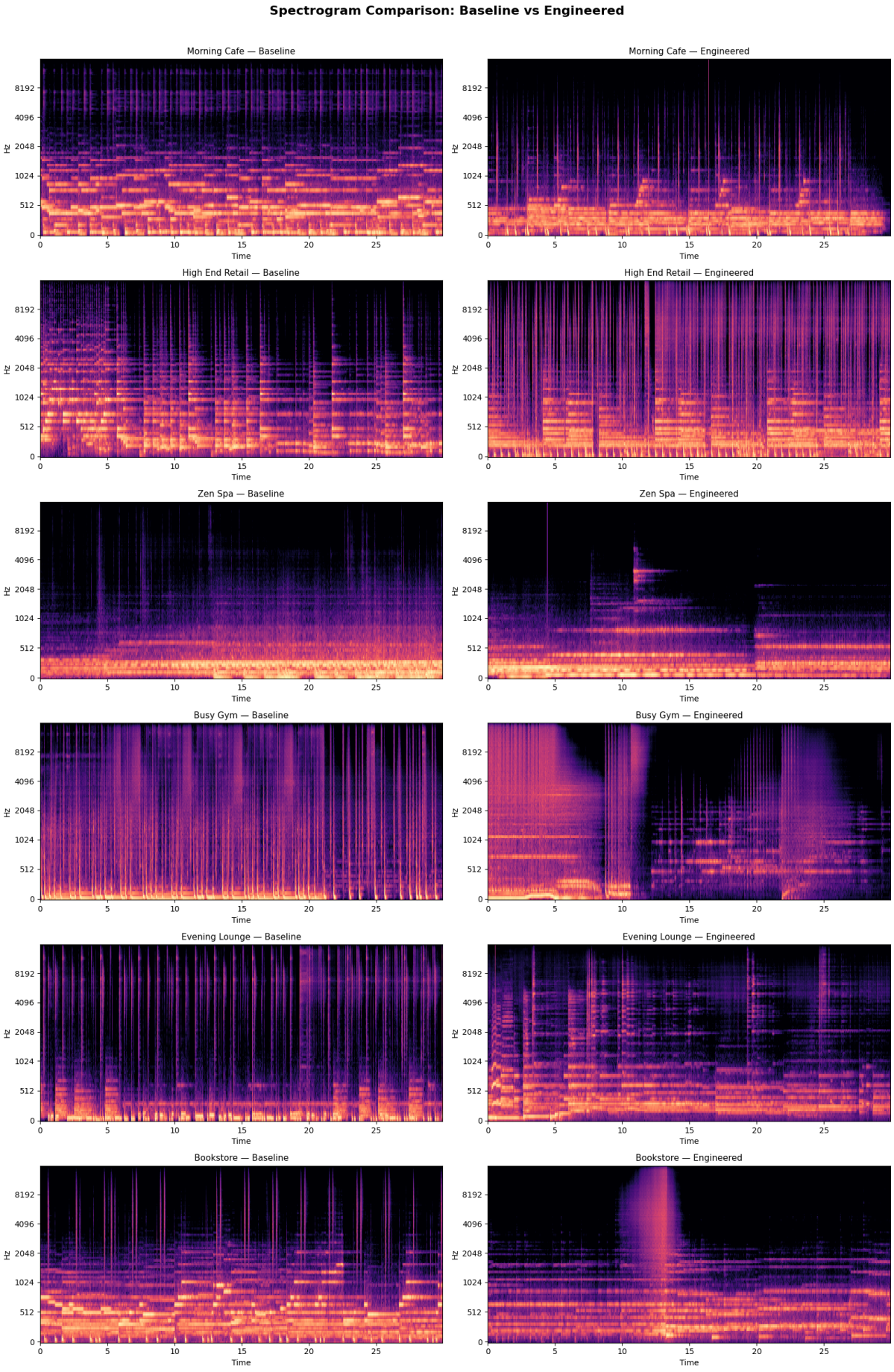

Figure 3: Spectrogram comparison showing frequency distribution differences between baseline and engineered prompts across all 6 vibes.

The spectrograms visually confirm that engineered prompts produce more structured, controlled frequency distributions tailored to each vibe's intended atmosphere.

7.4 Generation Latency

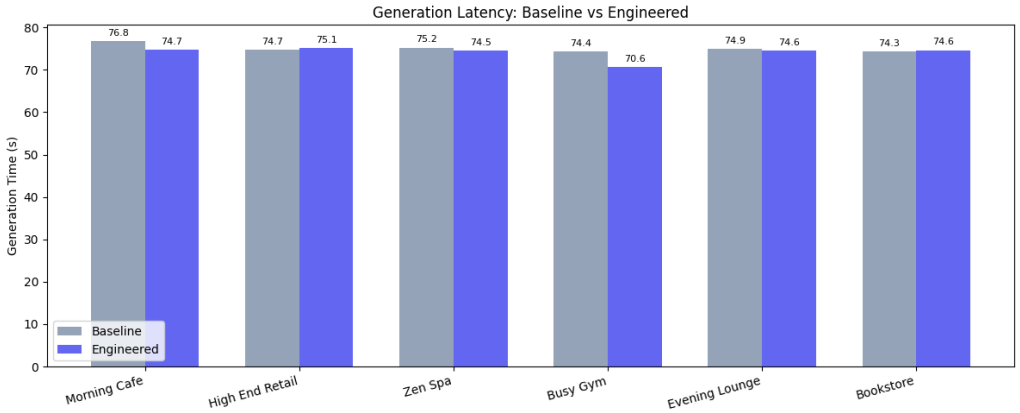

Figure 4: Generation time comparison: Baseline vs. Engineered latency for each vibe, averaging ∼74 seconds per 30-second clip on A100 GPU.

Generation latency is consistent at ∼74–75 seconds per 30-second clip on A100 GPU, with no significant difference between baseline and engineered prompts. This confirms that prompt complexity does not increase inference time.

7.5 Audio Quality Radar Charts

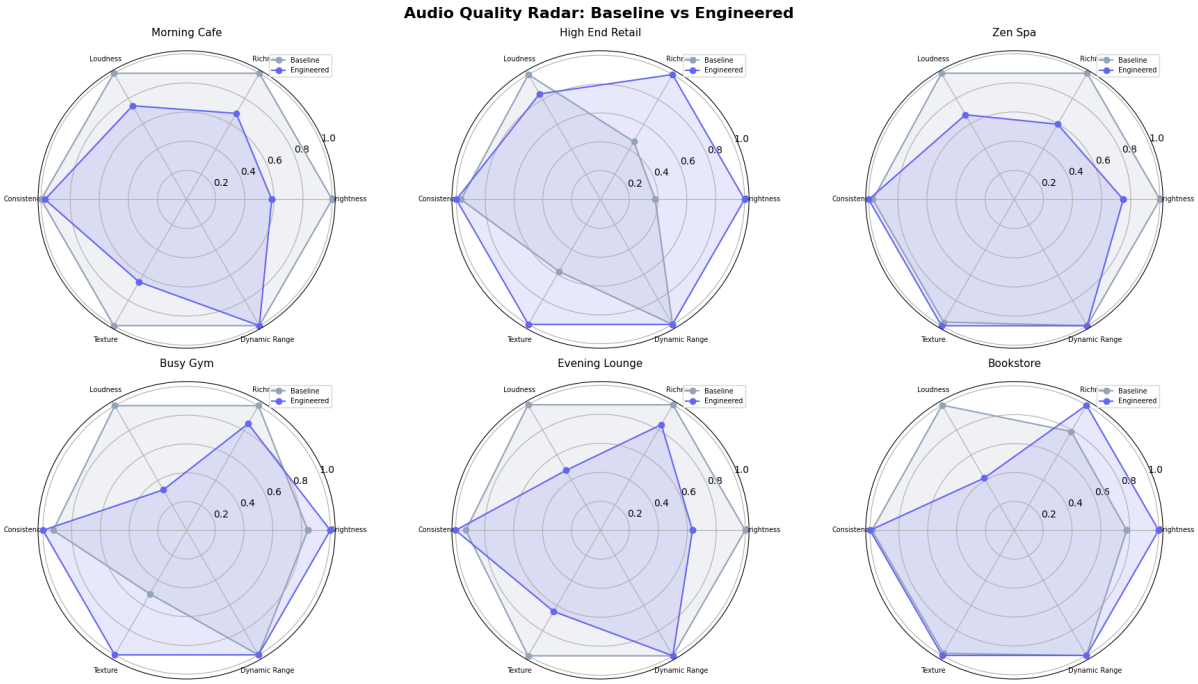

Figure 5: Radar charts comparing baseline vs. engineered audio quality across 6 dimensions: Loudness, Richness, Brightness, Dynamic Range, Texture, and Consistency.

The radar charts show that engineered prompts generally produce more balanced audio profiles, with improvements in consistency and texture while maintaining appropriate energy levels for each vibe type.

8 Technical Design Details

8.1 Audio Engine — Seamless Crossfade

The dual-track Howler.js engine eliminates gaps between clips:

```
Track A plays -> onplay pre-loads next clip from IndexedDB
  -> 26s in (30s - 4s fade) -> Track B fades in
  -> Track A fades out -> Track B onplay pre-loads next
  -> Infinite seamless loop
```

8.2 Background Scheduler — Priority System

The scheduler continuously generates clips with intelligent prioritization:

1. **If user is listening** → Always generate for the active vibe
2. **If idle** → Generate for the vibe with the fewest cached clips

8.3 IndexedDB Cache Architecture

All generated audio is persisted in the browser's IndexedDB:

- **Per-vibe storage** — Each clip tagged with its vibe ID
- **Duration calculation** — Parsed from WAV blob headers (32 kHz, 16-bit mono)
- **Per-vibe deletion** — Users can remove individual vibes and their cached audio
- **Survives page refresh** — Custom vibes persisted in localStorage, audio in IndexedDB

8.4 Custom Vibe Creation Flow

```
User Input: "rainy sunday bookstore"
       |
       v
Gemini 2.5 Flash -> Structured JSON:
  {
    name: "Rainy Bookstore Calm",
    icon: "book emoji",
    prompt: "A calm, slow-tempo, acoustic guitar and soft piano,
             gentle rain ambiance, no vocals, no lyrics, no singing,
             instrumental only, cozy bookstore atmosphere",
    gradient: "linear-gradient(135deg, #4a6741, #8b7355)"
  }
       |
       v
New vibe card added -> Scheduler starts generating
       |
       v
Audio plays with seamless crossfade
```

9 Limitations & Future Work

9.1 Current Limitations

1. **Generation Latency** — Each 30-second clip takes ∼74 seconds to generate on A100 GPU with MusicGen-Large. Users may experience a wait during cold start when no cached clips are available. MusicGen-Small is significantly faster but produces inferior audio quality unsuitable for professional use.
2. **Quality–Speed Trade-off** — MusicGen-Large produces high-quality audio but is inherently slow (∼2.5× real-time). There is no way to achieve both maximum quality and real-time generation with current hardware.
3. **No Real-Time Streaming** — Audio is generated in 30-second chunks, not as a true real-time stream. The crossfade system masks this limitation.
4. **Custom Vibe Quality** — Gemini prompt rewriting improves prompt quality, but MusicGen's adherence to complex prompts is imperfect. Some generated clips may not perfectly match the described atmosphere.
5. **Backend Dependency** — Requires an active Colab notebook with ngrok tunnel. The system cannot generate new clips if the backend is offline, though cached clips continue to play.

9.2 Future Work

6. **Cloud Database for Audio Storage** — Currently, all audio is stored locally in the browser's IndexedDB. In the future, I plan to migrate storage to a cloud database with per-user accounts, enabling users to access their music library from any device and eliminating browser storage limitations.
7. **Multi-Model Pipeline** — Explore using MusicGen-Small for quick previews and MusicGen-Large for final cached versions.
8. **Mood Scheduling** — Automatically adjust vibes based on time of day (morning → café, afternoon → retail, evening → lounge).

10 AI Tools Used

Table 5: AI Tools and Their Usage

Tool	How Used
Gemini (Antigravity)	Code assistance for frontend/backend development, debugging, and report writing
Google Gemini 2.5 Flash	Custom vibe prompt rewriting (in-app, via API)
Google Gemini 2.5 Flash TTS	Store announcement text-to-speech (in-app, via API)
Google Colab	GPU runtime for MusicGen model inference
Hugging Face Transformers	MusicGen model loading and inference

All core architecture decisions, prompt engineering design, and evaluation methodology were designed by the student. AI tools were used for code implementation acceleration and in-app features.

11 Repository Structure

```
AuraStream/
+-- notebooks/
|   +-- AuraStream_Colab.ipynb    # Model + Evaluation + API Server
+-- backend/
|   +-- app.py                     # FastAPI REST server
|   +-- generator.py               # MusicGen wrapper
|   +-- prompt_engine.py           # Prompt engineering logic
|   +-- requirements.txt
+-- frontend/
|   +-- index.html                 # Main UI
|   +-- css/style.css              # Design system
|   +-- js/
|       +-- app.js                 # Core logic, scheduler, Gemini
|       +-- audioEngine.js         # Howler.js crossfade engine
|       +-- visualizer.js          # Web Audio visualizer
+-- report_images/                 # Extracted notebook charts
+-- sample_outputs/                # Sample generated audio files
```

References

[1] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. *arXiv preprint arXiv:2306.05284*, 2023.

[2] Meta AI (FAIR). MusicGen-Large — 3.3B parameter text-to-music model. https://huggingface.co/facebook/musicgen-large, 2023.

[3] James Simpson. Howler.js — JavaScript audio library for the modern web. https://howlerjs.com/, 2023.

[4] Google DeepMind. Gemini 2.5 Flash — Multimodal AI model. https://deepmind.google/technologies/gemini/, 2025.